

Development Plan

Squiggly

Team #17, Squiggly
Rayan Nasrallah
Ahmed Elzaria
Muhammad Huzaifah
Shamil Canbolat
Moustafa Moustafa

Table 1: Revision History

Date	Developer(s)	Change
26 September 2026	Moustafa Moustafa	Added my reflection responses to the appendix.
26 September 2026	Rayan Nasrallah	Added reflection responses on the value of a development plan and on the advantages and disadvantages of CI/CD.

[Put your introductory blurb here. Often the blurb is a brief roadmap of what is contained in the report. —SS]

[Additional information on the development plan can be found in the [lecture slides](#). —SS]

1 Confidential Information?

[State whether your project has confidential information from industry, or not. If there is confidential information, point to the agreement you have in place. —SS]

[For most teams this section will just state that there is no confidential information to protect. —SS]

2 IP to Protect

[State whether there is IP to protect. If there is, point to the agreement. All students who are working on a project that requires an IP agreement are also required to sign the “Intellectual Property Guide Acknowledgement.” —SS]

3 Copyright License

[What copyright license is your team adopting. Point to the license in your repo. —SS]

4 Team Meeting Plan

[This section is for meetings between the team members, and meetings between the team and their supervisor/advisor (as appropriate). —SS]

[How often will the team meet? where? when? —SS]

[If the meeting is a physical location (not virtual), out of an abundance of caution for safety reasons you shouldn’t put the location online —SS]

[To help schedule meetings, you can use the lecture/tutorial time when we do not have anything else scheduled. —SS]

[How often will you meet with your supervisor/advisor? when? where? —SS]

[Will meetings be virtual? At least some meetings should likely be in-person. —SS]

[How will the meetings be structured? There should be a chair for all meetings. There should be an agenda for all meetings. —SS]

5 Team Communication Plan

[Issues on GitHub should be part of your communication plan. —SS]

6 Team Member Roles

[You should identify the types of roles you anticipate, like notetaker, leader, meeting chair, reviewer. Assigning specific people to those roles is not necessary at this stage. In a student team the role of the individuals will likely change throughout the year. —SS]

7 Workflow Plan

- How will you be using git, including branches, pull request, etc.?
- How will you be managing issues, including template issues, issue classification, etc.?
- Use of CI/CD

8 Project Decomposition and Scheduling

- How will you be using GitHub projects?
- Include a link to your GitHub project

[How will the project be scheduled? This is the big picture schedule, not details. You will need to reproduce information that is in the course outline for deadlines. —SS]

9 Proof of Concept Demonstration Plan

What is the main risk, or risks, for the success of your project? What will you demonstrate during your proof of concept demonstration to convince yourself that you will be able to overcome this risk?

10 Expected Technology

[What programming language or languages do you expect to use? What external libraries? What frameworks? What technologies. Are there major components of the implementation that you expect you will implement, despite the existence of libraries that provide the required functionality. For projects with machine learning, will you use pre-trained models, or be training your own model? —SS]

[The implementation decisions can, and likely will, change over the course of the project. The initial documentation should be written in an abstract way; it should be agnostic of the implementation choices, unless the implementation choices are project constraints. However, recording our initial thoughts on implementation helps understand the challenge level and feasibility of a project.

It may also help with early identification of areas where project members will need to augment their training. —SS]

Topics to discuss include the following:

- Specific programming language
- Specific libraries
- Pre-trained models
- Specific linter tool (if appropriate)
- Specific unit testing framework
- Investigation of code coverage measuring tools
- Specific plans for Continuous Integration (CI), or an explanation that CI is not being done
- Specific performance measuring tools (like Valgrind), if appropriate
- Tools you will likely be using?

[git, GitHub and GitHub projects should be part of your technology. —SS]

11 Use of AI and LLMs

[This section is a continuation of the previous section. The role of LLMs in your project as important enough to warrant their own section. How will your team be using LLMs for code and documentation development? What tools will you be using? How will you verify the output of your LLMs? —SS]

12 Coding Standard

[What coding standard will you adopt? —SS]

13 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

[This section is about the use of AI to write this document, not your planned use of AI for your project. You discuss that topic in one of the sections in the body of this report. —SS]

13.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

13.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

13.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

Appendix — Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. **Why is it important to create a development plan prior to starting the project?**

Ahmed Elzaria:

Creating the development plan helped me recognize that agreeing on a project idea does not automatically mean we agree on how to complete it. A team can share the same goals while having different expectations about responsibilities, deadlines, and acceptable contributions. Discussing these expectations early gives us a common basis for coordinating work and addressing problems before they affect a major deliverable.

While developing the team roles, I initially considered assigning each person a narrow area of implementation. However, I realized that this could restrict members' learning and create dependence on individuals. Making every responsibility shared introduced the opposite problem: accountability became unclear. The approach I settled on combines named leads with shared development duties and explicit task ownership. This showed me that planning involves balancing competing needs, rather than simply dividing a list of tasks.

The technology discussion also demonstrated the value of planning. Clarifying that enhanced drawings should remain editable changed what we needed from the implementation: returning a generated image would not satisfy that expectation. Identifying this distinction early helped expose a feasibility question before investing in an unsuitable approach. I therefore see the development plan as a starting point that makes assumptions visible and guides early investigation, while remaining open to revision as we learn.

Moustafa Moustafa:

A development plan makes it clear who is responsible for what before the work starts. Without one, tasks are unclear and it is easy for someone to say "I don't want to do this" with nothing to point back to. When we

define roles, who works on what, how we split tasks, and which tools each person uses, it saves a lot of friction later.

I have learned this from experience. On a previous project, we did not have a well-defined plan, and it came back to haunt us near the end. All three of us got busy with university just as some milestones were due. Nobody wanted to take them on, but someone had to, and the work ended up being split unevenly across 2 people. With a clear plan, clearly separated tasks, and agreed roles, we could have avoided that situation entirely. That is why I think having the plan in place from the start matters so much for a five-person capstone team.

Rayan Nasrallah:

The benefit I found clearest was being made to sketch the technical implementation at a high level, because doing so exposed what we did not yet know how to build. Committing to a rough shape for the system forced questions that a feature list never raises on its own. Our discussion about keeping refined drawings editable is a good example: the requirement sounds straightforward until you consider the implementation, at which point returning a generated image clearly fails to satisfy it. Identifying that early gives us time to investigate, rather than discovering the problem once work depends on the wrong approach.

These feasibility questions feed directly into scope, which matters more here than it would on an open-ended project. We have two semesters, five members, and a fixed set of deliverables alongside the implementation, so the amount of work a feature requires is as important as how valuable it is. Thinking this through in advance is what let us treat structured practice, adaptive guidance, and progress reporting as core work while moving camera-based tracing and letter activities to stretch goals. Without estimating the effort first, that judgement would have been guesswork.

I also think there is a real difference between discussing these decisions and writing them down. Discussion can produce the feeling of agreement while members leave with different understandings of what was decided, and writing forces the vagueness to surface. The written plan also gives us a concrete reference to revise. I do expect parts of it to prove wrong once implementation begins, since we are planning around technology we have not used yet, so I see its value less in being accurate and more in making our assumptions explicit enough to correct.

2. In your opinion, what are the advantages and disadvantages of using CI/CD?

Ahmed Elzaria:

I see the main advantage of Continuous Integration (CI) as giving the team consistent feedback whenever changes are proposed. With five members contributing, relying on each person to remember and reproduce every check would make mistakes easier to miss. Automated builds, linting, and

tests can identify problems while the relevant change is still small and easier to investigate. For our documentation, an automated $\text{\LaTeX}$ build can also reveal compilation errors before submission.

Continuous Delivery or Deployment (CD) extends this process. Continuous delivery keeps changes ready for release, while continuous deployment automatically releases changes that pass the required checks. For Squigglr, preview deployments would make it easier for teammates to review an interface change and try it on a tablet. However, passing automated checks should not be treated as proof that a change is ready for users. Drawing responsiveness and stylus behaviour still need evaluation on actual devices, and a successful document build does not establish that its content satisfies the rubric.

The main disadvantages are the effort required to configure and maintain the workflows, the time spent investigating failed runs, and the risk of trusting incomplete tests too much. A pipeline can pass while important behaviour remains untested. For example, a scoring function could produce consistently incorrect results if its tests use the same mistaken assumptions as its implementation.

Given our timeline, I favour introducing a small set of useful checks first and expanding them as the application develops. This would allow us to gain early feedback without spending disproportionate effort on automation before the core functionality exists. Human review and device testing would remain part of our process.

Moustafa Moustafa:

I first saw the real value of CI/CD during my previous co-op. It was my first experience on a large project, and I realized that a codebase of that size could not function without it once more than a few people were contributing. The same applies on a smaller scale to capstone. With five people working in the same repository, we would constantly run into conflicts and broken builds without CI/CD, and it would be hard to tell which change caused a problem. Automated checks on every pull request catch issues early, make it clear which change introduced them, and make it much easier to roll back when something goes wrong.

The main disadvantage is that it can become a bit of a rabbit hole. Some people keep improving their pipeline, adding more optimizations and tools, until maintaining the CI/CD takes more time than the actual work. There is a balance to find: the pipeline should be good enough to let the team work together safely and recover from mistakes, without being over-engineered. For us, that means keeping CI/CD simple and only adding to it when we actually need to.

Rayan Nasrallah:

Continuous Integration means merging our work into the shared branch frequently, with automated checks running on it each time, instead of integrating large amounts of work at once. Continuous Delivery keeps every

change that passes those checks ready to release, while continuous deployment releases it without waiting for a person. Our repository already does all of this for documentation: a GitHub Actions workflow determines which documents a change affects, compiles them, commits the resulting PDFs, and publishes them to our project page. Once implementation begins, the same pipeline would run tests and linting rather than only document builds.

The advantage I value most is the effect this has on how we work, rather than what it automates. Because every change arrives as a pull request that has to build before it can merge, work cannot sit unintegrated on a branch for weeks and then arrive all at once near a deadline. Changes stay small enough for somebody other than the author to read properly, and the project advances in increments we can each follow.

The published output stays current for the same reason. Our PDFs always correspond to what is on the main branch, so nobody has to compile everything before a submission and discover a problem then. For the application, preview deployments would let the rest of the team try an interface change on a tablet instead of taking the author's description of it on trust.

I initially thought there were no real disadvantages, and looking more carefully at our own workflow changed that view. The clearest cost is that a passing pipeline is easy to mistake for a correct result. Our problem statement compiled without error while citing references the document did not yet contain, because L^AT_EX has no reason to object to that. The same failure will be available to us in code, where tests can pass while drawing responsiveness on a real tablet remains wrong. Automated checks only cover what we thought to check, so I read a passing run as evidence that nothing obvious broke rather than that the work is right.

3. What disagreements did your group have in this deliverable, if any, and how did you resolve them?

Ahmed Elzaria:

Our main differences concerned the consequences of missing meetings and the roles and expectations of each team member. These discussions required more communication than simply dividing the deliverable because they established how we would hold one another accountable throughout the project.

For attendance and missed commitments, I wanted expectations that were clear enough to act on while recognizing that illness, emergencies, and unavoidable conflicts can occur. I saw a risk in both extremes: vague consequences could allow repeated problems to continue, while an overly rigid response could treat a genuine difficulty as a lack of effort. Through discussion, we developed an approach with measurable triggers and a staged response. The charter begins with a private conversation and a recovery

plan, followed by escalation to the teaching team if commitments remain unmet without an acceptable explanation. This gives the team a basis for taking action while allowing us to understand the circumstances first.

We also differed over how specifically to define individual roles. While working on this section, I found that narrow assignments could make members feel restricted to one component, whereas making everything a shared responsibility left no clear person to coordinate the work. The resulting structure assigns broad lead roles while making software development, testing, documentation, and peer review responsibilities of every member. Assigning Moustafa as the notetaker provides a consistent owner for meeting records, while rotating the quality assurance coordinator each milestone distributes deliverable review duties.

These discussions changed how I understood fairness within the team. Giving everyone the same number of titles does not necessarily create an equal workload, because implementation, coordination, and review require different amounts of effort at different stages. I now see the role assignments as a starting point that must be supported by clear task ownership and regular workload reviews.

My main lesson was that expectations need to be discussed explicitly rather than assumed. Writing down the consequences, role boundaries, and shared duties helped turn our differing views into a practical agreement. However, agreement on the document does not guarantee that the process will work well. We will need to revisit it as the workload increases and check whether the responsibilities and responses remain fair and useful.

Appendix — Team Charter

[borrows from [University of Portland Team Charter](#) —SS]

External Goals

[What are your team’s external goals for this project? These are not the goals related to the functionality or quality of the project. These are the goals on what the team wishes to achieve with the project. Potential goals are to win a prize at the Capstone EXPO, or to have something to talk about in interviews, or to get an A+, etc. —SS]

Attendance

Expectations

[What are your team’s expectations regarding meeting attendance (being on time, leaving early, missing meetings, etc.)? —SS]

Acceptable Excuse

[What constitutes an acceptable excuse for missing a meeting or a deadline? What types of excuses will not be considered acceptable? —SS]

In Case of Emergency

[What process will team members follow if they have an emergency and cannot attend a team meeting or complete their individual work promised for a team deliverable? —SS]

Accountability and Teamwork

Quality

[What are your team’s expectations regarding the quality of team members’ preparation for team meetings and the quality of the deliverables that members bring to the team? —SS]

Attitude

[What are your team’s expectations regarding team members’ ideas, interactions with the team, cooperation, attitudes, and anything else regarding team member contributions? Do you want to introduce a code of conduct? Do you want a conflict resolution plan? Can adopt existing codes of conduct. —SS]

Stay on Track

[What methods will be used to keep the team on track? How will your team ensure that members contribute as expected to the team and that the team performs as expected? How will your team reward members who do well and manage members whose performance is below expectations? What are the consequences for someone not contributing their fair share? —SS]

[You may wish to use the project management metrics collected for the TA and instructor for this. —SS]

[You can set target metrics for attendance, commits, etc. What are the consequences if someone doesn't hit their targets? Do they need to bring the coffee to the next team meeting? Does the team need to make an appointment with their TA, or the instructor? Are there incentives for reaching targets early? —SS]

Team Building

[How will you build team cohesion (fun time, group rituals, etc.)? —SS]

Decision Making

[How will you make decisions in your group? Consensus? Vote? How will you handle disagreements? —SS]